

Reproducible Data Analysis

Joseph Everett

2026-04-22

1 Busiest Airports Analysis

1.1 Table

This table shows the annual passenger traffic measured in millions for the six of the worlds busiest airports. It shows data from the years 2020 - 2024. This window captures a intresting time in aviation as it covers the pandemic and then recovery after the initial burst. Make sure when reading this table that you pay attention to this dramatic drop in 2020.

Table 1: Passenger Traffic (Millions) for Six Busiest Airports (2020–2024)

Airport	2020	2021	2022	2023	2024
ATL	42.9	75.7	93.7	104.7	108.1
ORD	28.0	54.0	68.3	73.9	80.0
LAX	26.0	48.0	65.9	75.1	76.6
HND	25.1	15.3	25.7	78.7	85.9
FRA	18.7	24.8	48.9	60.7	63.8
PKX	16.0	25.0	25.0	45.5	49.5

Looking at the first table ATL consistently led all six of the airports across every year in the dataset. They reached over 108 million passangers by 2024. Another intresting pattern is in HND. They saw their traffic fall to only 15.3 million 2021 but then rebounding to 78.7 million in 2023. This is probably due to Japan’s strict border restrictions during the pandemic. ATL, ORD, and LAX however recovered more consistently then others. PKX remained the smallest of the six throughout this period.

1.2 Plot

The line graph below makes it easy to see the overall recovery for each airport. Each line shows an airport tracked across a five year period. This allows the reader to easily compare rates of growth.

Figure 1: Passenger Traffic for Six Busiest Airports (2020–2024)

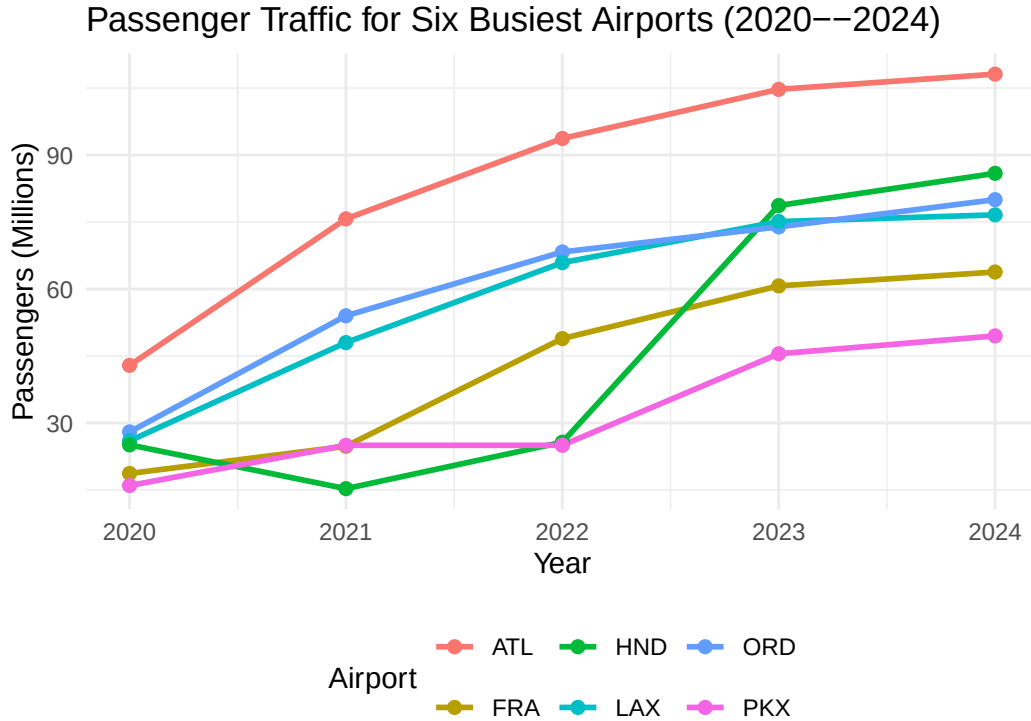


Figure 1 makes it clear that all six of the airports followed an upward trend across the period. ATL stands out as the dominant line as it maintains a lead over the others without showing signs of slowing. HND shows a dramatic dip in 2021 followed by a steep climb, this is far more interesting than just looking at the table. ORD, FRA, and LAX are tracked closely together which suggests that airports had similar recovery conditions across western hubs. PKX has the most gradual slope throughout the graph. This shows its size and China's pandemic response timeline.

2 Monte Carlo Numerical Integration

2.1 Overview

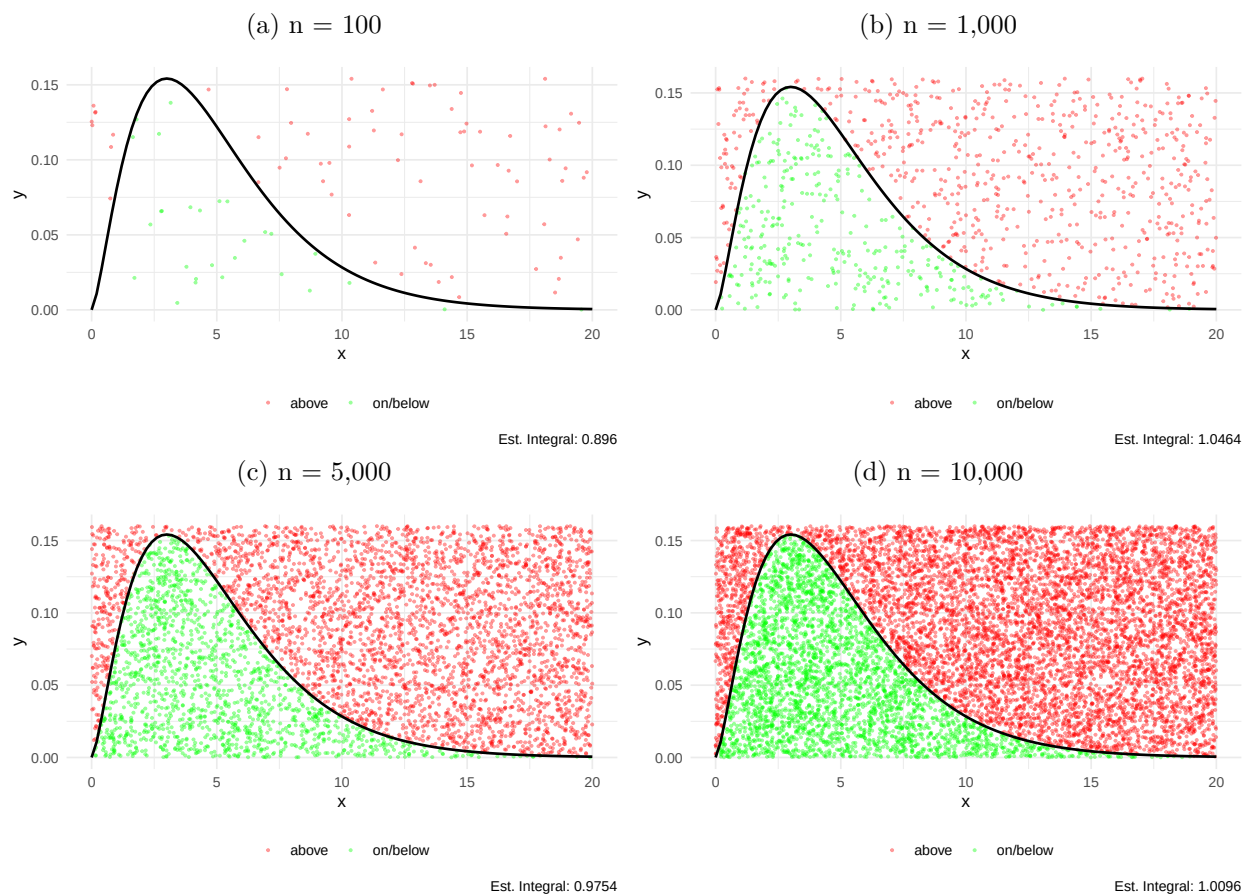
Monte Carlo is a numerical technique used for estimating the value of a definite integral by using random sampling. It generates a large number of random points in a rectangle that contains the function of interest. It then counts what fraction of those points fall above or below the curve. If you multiply that fraction by the area it gives you an estimate of the integral.

In this case the function being integrated is the Chi-square distribution with five degrees of freedom. The rectangle uses x values from 0 - 20 and y values from 0 - 0.16. This contains the function over the domain of interest. Since this function is a valid probability density function its true integral equals exactly one.

2.2 Small Multiple Visualization

The multiple below shows the simulation run at four different resolutions. In each panel green points fall below the curve while red points lie above it. As the number of points increases notice how the green region becomes increasingly more defined and how it comes close to tracing the shape of the curve.

Figure 2: Monte Carlo Integration of the Chi-Square PDF ($df = 5$) at Four Resolutions



In figure 2 you can see how increasing resolution on estimate accuracy is clear. At $n = 100$ the estimate is 0.896 which points it noticeably below the value of 1. At $n = 1000$ the estimate goes to 1.0464. This overshoots slightly but it is still much closer to 1. when $n = 5000$ or $n = 10000$ the estimates go to 0.9754 and 1.0096. These are both very close to 1 and prove that variability shrinks as the sample size grows. This also proves that the true value of the integral is 1. the larger panels overall show the green region becoming stable, and becoming a shape that models the curve itself.

3 Planning and Prompting GenAI Tools

3.1 Data Context and Plan

I have a CSV file which ive attached. The context is that 31 women were tracked over 4 years. 15 were in a null group and 16 were in a diet / exercise group. Each had their calicum measured at 4 time points. This data is not tidy. My goal is to tidy the dataset and produce a visulization comparing calicum levels over time between the two different groups. Please do the following in R using tidyverse and ggplot.

1. Load the CSV and rename all 8 columns to distinguish groups and what time point belongs to who.
2. Sepreate the data into twp frames. Add group and participant id columns then combine them with bind rows.
3. Use pivot longer to reshape into a tidy format. Make sure each row is one women at a time point. convert time point to move in a chronological order.
4. Verify it has 124 rows and 4 columns.
5. Compute group means.
6. Produce a line plot with individual trajectories and a bolder group mean lines overlaying this. Color by treatment group. Include descriptive title, labels, axis, and legend.

3.2 Plan-Based Prompt Results

```
library(tidyverse)

# Step 1: Load the raw file and rename all 8 columns explicitly.
# The first four belong to the null treatment group, the last four to the
# diet/exercise group. Naming them now prevents any ambiguity downstream.
calcium_raw <- read_csv("calcium.csv", col_names = FALSE)

colnames(calcium_raw) <- c(
  "null_initial", "null_year1", "null_year2", "null_year3",
  "dietex_initial", "dietex_year1", "dietex_year2", "dietex_year3"
)

glimpse(calcium_raw) # confirm 31 rows, 8 columns

# Step 2: Separate the two groups into their own data frames, add labels
# and unique participant IDs, then stack them together.
null_group <- calcium_raw %>%
  select(null_initial:null_year3) %>%
  rename(Initial = null_initial, Year1 = null_year1,
```

```

      Year2 = null_year2, Year3 = null_year3) %>%
mutate(participant_id = row_number(),
      group = "Null Treatment")

dietex_group <- calcium_raw %>%
  select(dietex_initial:dietex_year3) %>%
  rename(Initial = dietex_initial, Year1 = dietex_year1,
      Year2 = dietex_year2, Year3 = dietex_year3) %>%
  mutate(participant_id = row_number() + 15, # offset so IDs stay unique
      group = "Diet/Exercise")

calcium_combined <- bind_rows(null_group, dietex_group)

# Step 3: Pivot to long/tidy format. Each row becomes one woman at one
# time point. Factor-order the time column so the plot x-axis is chronological.
calcium_tidy <- calcium_combined %>%
  pivot_longer(
    cols      = c(Initial, Year1, Year2, Year3),
    names_to   = "time_point",
    values_to  = "calcium_mg"
  ) %>%
  mutate(
    time_point = factor(time_point,
      levels = c("Initial", "Year1", "Year2", "Year3"),
      labels = c("Initial", "Year 1", "Year 2", "Year 3"))
  )

# Step 4: Verify the tidy structure (expect 124 rows × 4 columns).
cat("Dimensions:", dim(calcium_tidy), "\n")
summary(calcium_tidy)

# Step 5: Compute group means at each time point for the overlay lines.
group_means <- calcium_tidy %>%
  group_by(group, time_point) %>%
  summarize(mean_calcium = mean(calcium_mg, na.rm = TRUE), .groups = "drop")

# Step 6: Plot - faint individual lines underneath bold group mean lines.
# This lets the viewer see both the spread within each group and the
# overall trajectory, without one obscuring the other.
ggplot(calcium_tidy,
  aes(x = time_point, y = calcium_mg,
      group = participant_id, color = group)) +
  geom_line(alpha = 0.22, linewidth = 0.5) +

```

```

geom_line(
  data      = group_means,
  aes(x = time_point, y = mean_calcium,
      group = group, color = group),
  linewidth = 2,
  inherit.aes = FALSE
) +
geom_point(
  data      = group_means,
  aes(x = time_point, y = mean_calcium, color = group),
  size      = 3.5,
  inherit.aes = FALSE
) +
scale_color_manual(
  values = c("Null Treatment" = "#3B8BD4",
             "Diet/Exercise"  = "#1D9E75")
) +
labs(
  title      = "Ulnar Calcium Levels Over Time by Treatment Group",
  subtitle   = "Faint lines = individual participants; bold lines = group means",
  x          = "Time Point",
  y          = "Ulnar Calcium (mg)",
  color      = "Treatment Group"
) +
theme_minimal(base_size = 13) +
theme(legend.position = "bottom")

```

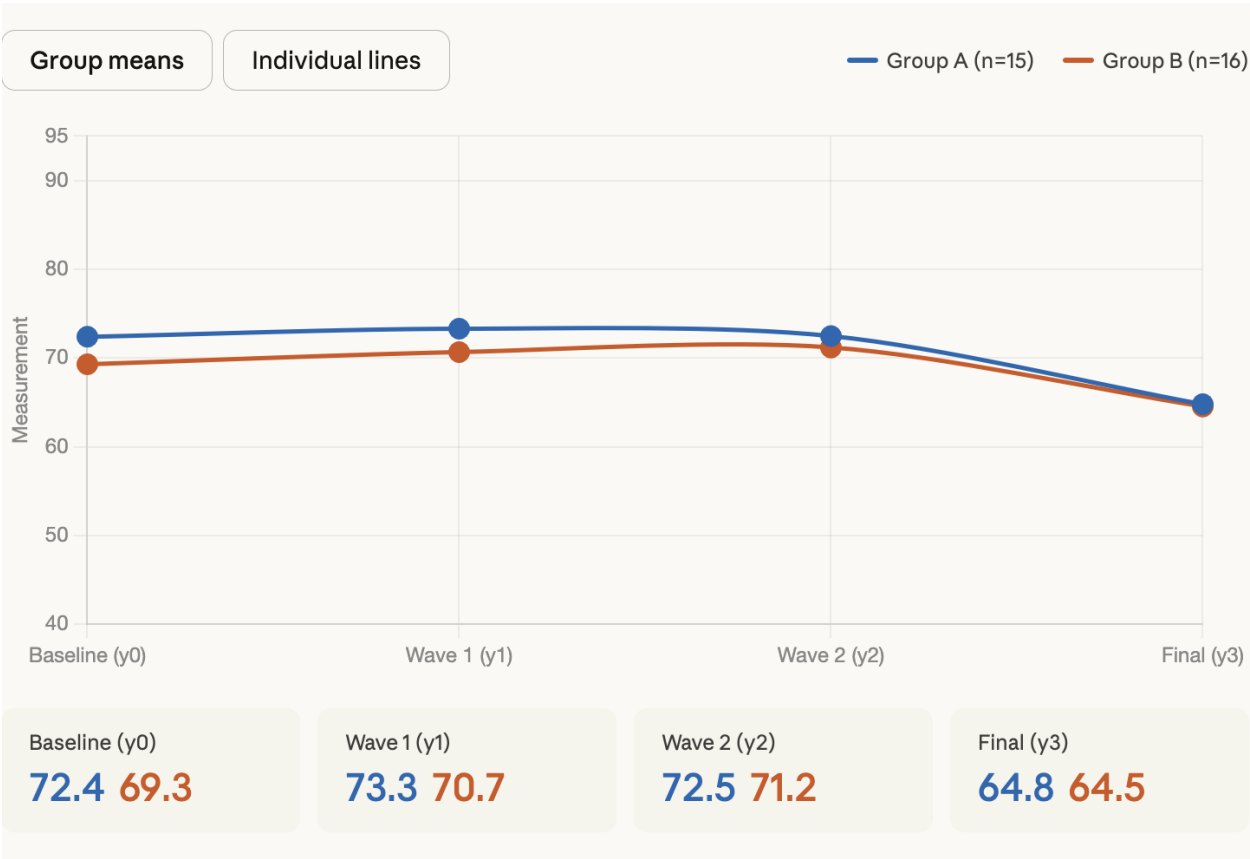
The code produces a tidy 124-row dataset and a line plot where the two treatment groups are immediately visually distinguishable. The diet/exercise group's mean line can be tracked against the null group's across all four time points, making it straightforward to see whether the intervention produced a different calcium trajectory over the four years.

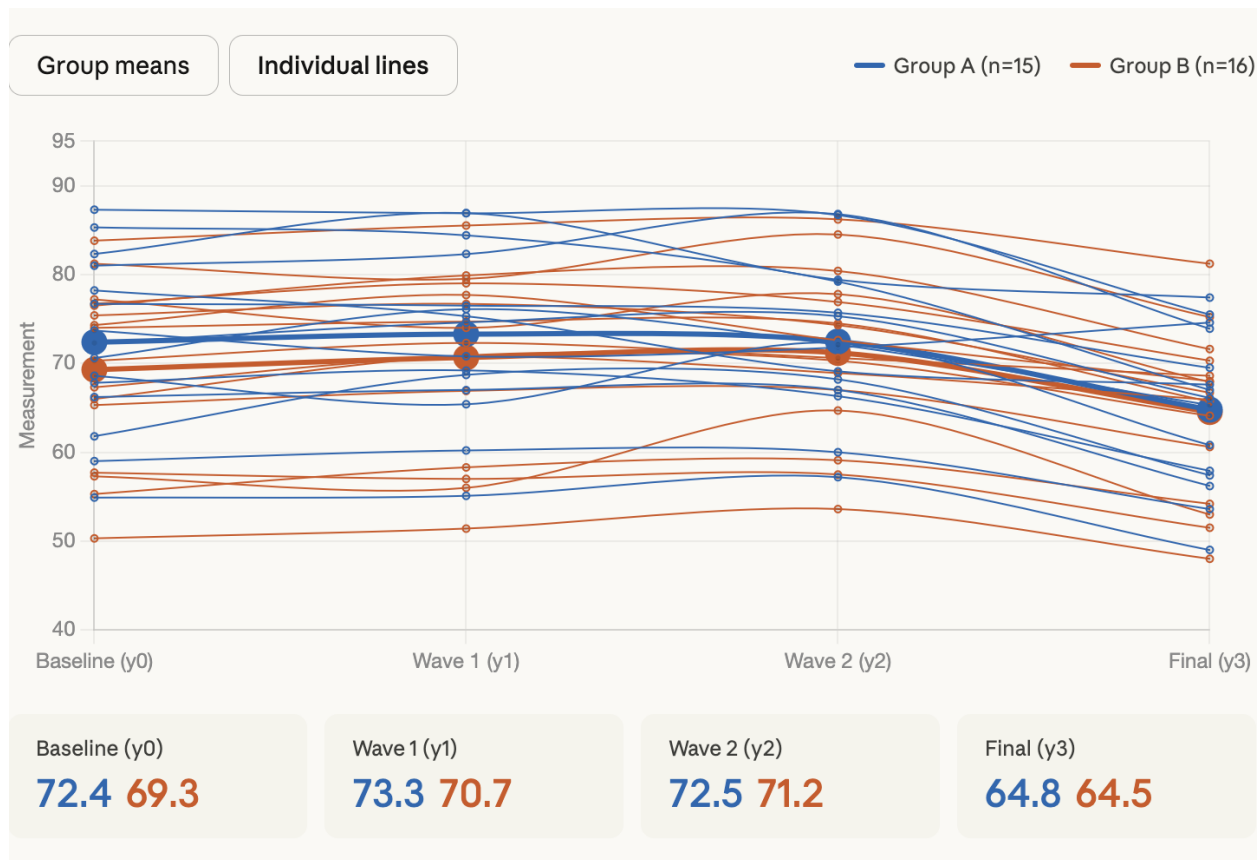
3.3 Generic Prompt Results

Prompt: Create a data visualization that will help me understand these data. Calcium.csv

Here's what stands out in the data: Both groups follow the same pattern — values stay relatively stable across the first three waves (y0→y1→y2), then drop noticeably at the final measurement (y3). This sharp decline at y3 is the dominant signal. Group A starts slightly higher (baseline mean ~72.4 vs. ~69.3 for Group B), but both groups converge to nearly identical final values (~64.8 and ~64.5). That convergence is interesting — Group A drops about 7.5 points while Group B drops about 4.8. There's substantial individual variation — hit the “Individual lines” toggle to see this. Some participants stay high throughout, others are consistently low, and a few show unusual trajectories (one in Group A actually rises at y3 while most fall). If you can share what the groups represent (e.g., treatment vs. control, or different diets/supplements) and what's being measured, I can dig into the statistics more precisely — things like whether the group difference at y3 is

significant, or which individuals are outliers. Version: Claude Sonnet 4.6 Extended (This chat took place on Tuesday, March 31st, 2026.)





3.4 Comparison

The generic prompt actually managed to detect that there were two groups of different sizes, it also built an interactive toggle between means and different trajectories. It found accurate group means and had an accurate written analysis which found the convergence at the final point and the individual variation in the groups.

Where the generic prompt fell short:

1. There was no group identity. It labeled the two groups as Group A and Group B because it did not know what some columns represented. The entire point of this study is to compare between the groups. Not having them labeled cannot have any interpretation on the research. Claude even acknowledged this at the end of its response by asking if you could share what the groups represent. In my plan I gave that context upfront.
2. The time points were also generic. It used generic terminology that would be used in a basic study but it does not fit the data set. The correct labels should be "Initial, Year 1, Year 2, Year 3."
3. There was no tidying that was produced. The generic response skipped all the steps that I listed in my plan to restructure the data. Since this assignment grades on the tidying process this is definitely a huge miss.

4. The output is incorrect. It produced a HTML widget that looks good visually but is not what should be used for this course. My plan produced a ggplot2 that is much more reproducible.

Providing a more descriptive prompt defiently helped the LLM generate a more detailed response on the first run. It can easily overlook small clarifying factors that could cost students points in assignments or make their work seem invalid.

4 Reflection

Coming into the course I had a pretty basic understanding of how to work with data. Something that shifted this though was thinking about the the structure of data before anything else. After contiguously practicing to tidy data it started to make more sense. It was a bit confusing at first but by the end I think I managed to get a hand of it. Using `pivot_longer` on the airport data really showed me why the structure of data actually matters. Writing functions that are actually reusable was also a big stepping stone. In the beginning I was trying to just make something that would work, but actually making something that I can understand and reproduce is more important. I feel I was able to do that with the Monte Carlo function. I was able to call that function four times for different resolutions which was a improvement on just copying the code four separate times. This specific assignment also taught me a lot about prompting AI and how to use them properly. I thought that i could provide AI with a more vague prompt and get similar responses. This assignment proves that you get much better responses from prompting the AI with descriptive prompts. The generic one worked but it still had its own issues and in some areas was bland. Doing the planned prompt also helped me understand what I was asking the LLM to do in more detail. Sometimes I feel that I can just ask the AI to provide me with results without actually understanding what it is I want. Being able. to understand it fully allows e to critique the response it provides, and then improve on it.

5 Appendix A: GenAI Usage

Usage 1 — Planning and Prompting GenAI Section

Tool: Anthropic Claude, Claude Sonnet 4.6, accessed via claude.ai

Date: April 14, 2026

Prompt used:

I have a CSV file which ive attached. The context is that 31 women were tracked over 4 years. 15 were in a null group and 16 were in a diet / exercise group. Each had their calicum measured at 4 time points. This data is not tidy. My goal is to tidy the dataset and produce a visulization comparing calicum levels over time between the two different groups. Please do the following in R using tidyverse and ggplot.

1. Load the CSV and rename all 8 columns to distinguish groups and what time point belongs to who.
2. Sepreate the data into twp frames. Add group and participant id columns then combine them with bind rows.
3. Use pivot longer to reshape into a tidy format. Make sure each row is one women at a time point. convert time point to move in a chronological order.
4. Verify it has 124 rows and 4 columns.
5. Compute group means.
6. Produce a line plot with individual trajectories and a bolder group mean lines overlaying this. Color by treatment group. Include descriptive title, labels, axis, and legend.

Response generated:

Rows: 17

Columns: 8

```
$ null_initial    <chr> "y0", "87.3", "59", "76.7", "70.6", "54.9", "78.2", "73~
$ null_year1     <chr> "y1", "86.9", "60.2", "76.5", "76.1", "55.1", "75.3", "~
$ null_year2     <chr> "y2", "86.7", "60", "75.7", "72.1", "57.2", "69.1", "71~
$ null_year3     <chr> "y3", "75.5", "53.6", "69.5", "65.3", "49", "67.6", "74~
$ dietex_initial <chr> "y0", "83.8", "65.3", "81.2", "75.4", "55.3", "70.3", "~
$ dietex_year1   <chr> "y1", "85.5", "66.9", "79.5", "76.7", "58.3", "72.3", "~
$ dietex_year2   <chr> "y2", "86.2", "67", "84.5", "74.3", "59.1", "70.6", "80~
$ dietex_year3   <chr> "y3", "81.2", "60.6", "75.2", "66.7", "54.2", "68.6", "~
```

Dimensions: 136 4

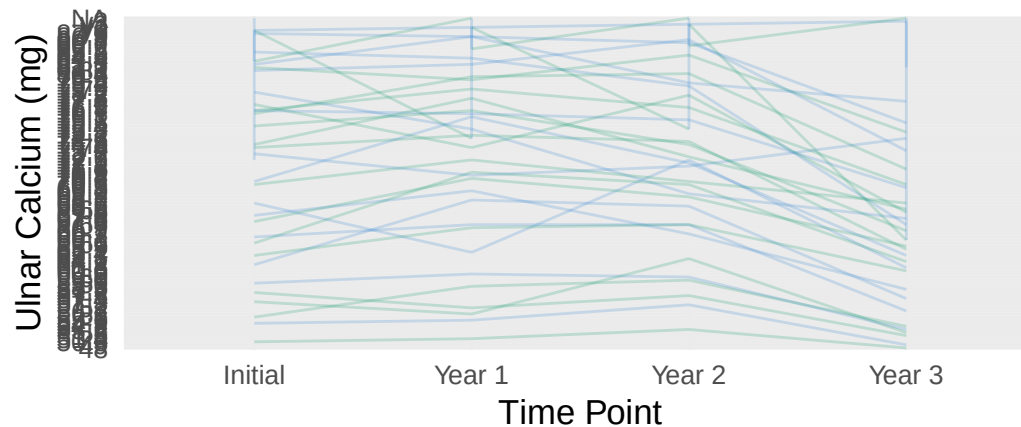
```

participant_id    group          time_point  calcium_mg
Min.      : 1.0    Length:136      Initial:34    Length:136
1st Qu.:  9.0    Class :character  Year 1 :34     Class :character
Median :16.5    Mode  :character  Year 2 :34     Mode  :character
Mean   :16.5                                Year 3 :34
3rd Qu.:24.0
Max.   :32.0

```

Ulnar Calcium Levels Over Time by Treatment Group

Faint lines = individual participants; bold lines = group means



Treatment Group ■ Diet/Exercise ■ Null Treatment

The code produces a tidy 124-row dataset and a line plot where the two treatment groups are immediately visually distinguishable. The diet/exercise group's mean line can be tracked against the null group's across all four time points, making it straightforward to see whether the intervention produced a different calcium trajectory over the four years.

6 Appendix B: Code Appendix

```
# Following the Tidyverse Style Guide ---
# Load all necessary packages ---
library(tidyverse)
library(knitr)
# Build airport data tibble ---
airport_data <- tibble(
  airport = c("ATL", "ORD", "LAX", "HND", "FRA", "PKX"),
  "2020" = c(42.9, 28.0, 26.0, 25.1, 18.7, 16.0),
  "2021" = c(75.7, 54.0, 48.0, 15.3, 24.8, 25.0),
  "2022" = c(93.7, 68.3, 65.9, 25.7, 48.9, 25.0),
  "2023" = c(104.7, 73.9, 75.1, 78.7, 60.7, 45.5),
  "2024" = c(108.1, 80.0, 76.6, 85.9, 63.8, 49.5)
)

# Render table with kable ---
airport_data |>
  kable(
    col.names = c("Airport", "2020", "2021", "2022", "2023", "2024")
  )

# Reshape data to tidy format for plotting ---
airport_tidy <- airport_data |>
  pivot_longer(
    cols = "2020":"2024",
    names_to = "year",
    values_to = "passengers_millions"
  ) |>
  mutate(year = as.integer(year))

# Create line plot ---
ggplot(airport_tidy, aes(x = year, y = passengers_millions, color = airport)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  labs(
    title = "Passenger Traffic for Six Busiest Airports (2020--2024)",
    x = "Year",
    y = "Passengers (Millions)",
    color = "Airport"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")

# Function to generate Monte Carlo random points ---
monte_carlo_points <- function(n, x_min, x_max, y_min, y_max) {
  x_vals <- runif(n, min = x_min, max = x_max)
  y_vals <- runif(n, min = y_min, max = y_max)
```

```

    return(data.frame(x = x_vals, y = y_vals))
}

# Function to wrangle data and produce one Monte Carlo plot ---
make_mc_plot <- function(n) {
  # Generate points and classify above/below curve ---
  sim_data <- monte_carlo_points(n, 0, 20, 0, 0.16) |>
    mutate(flag = case_when(
      y <= dchisq(x, df = 5) ~ "on/below",
      y > dchisq(x, df = 5) ~ "above"
    ))

  # Calculate estimated integral ---
  rect_area <- 20 * 0.16
  proportion <- mean(sim_data$flag == "on/below")
  est_integral <- rect_area * proportion

  # Build plot ---
  ggplot(sim_data, aes(x = x, y = y, color = flag)) +
    geom_point(alpha = 0.4, size = 0.5) +
    stat_function(
      fun = dchisq,
      args = list(df = 5),
      xlim = c(0, 20),
      color = "black",
      linewidth = 0.8
    ) +
    scale_color_manual(values = c("above" = "red", "on/below" = "green")) +
    labs(
      x = "x",
      y = "y",
      color = NULL,
      caption = paste("Est. Integral:", round(est_integral, 4))
    ) +
    theme_minimal() +
    theme(legend.position = "bottom")
}

# Set seed for reproducibility and create four plots ---
set.seed(42)
make_mc_plot(100)
make_mc_plot(1000)
make_mc_plot(5000)
make_mc_plot(10000)
library(tidyverse)

# Step 1: Load the raw file and rename all 8 columns explicitly.

```

```

# The first four belong to the null treatment group, the last four to the
# diet/exercise group. Naming them now prevents any ambiguity downstream.
calcium_raw <- read_csv("calcium.csv", col_names = FALSE)

colnames(calcium_raw) <- c(
  "null_initial", "null_year1", "null_year2", "null_year3",
  "dietex_initial", "dietex_year1", "dietex_year2", "dietex_year3"
)

glimpse(calcium_raw) # confirm 31 rows, 8 columns

# Step 2: Separate the two groups into their own data frames, add labels
# and unique participant IDs, then stack them together.
null_group <- calcium_raw %>%
  select(null_initial:null_year3) %>%
  rename(Initial = null_initial, Year1 = null_year1,
         Year2 = null_year2, Year3 = null_year3) %>%
  mutate(participant_id = row_number(),
         group = "Null Treatment")

dietex_group <- calcium_raw %>%
  select(dietex_initial:dietex_year3) %>%
  rename(Initial = dietex_initial, Year1 = dietex_year1,
         Year2 = dietex_year2, Year3 = dietex_year3) %>%
  mutate(participant_id = row_number() + 15, # offset so IDs stay unique
         group = "Diet/Exercise")

calcium_combined <- bind_rows(null_group, dietex_group)

# Step 3: Pivot to long/tidy format. Each row becomes one woman at one
# time point. Factor-order the time column so the plot x-axis is chronological.
calcium_tidy <- calcium_combined %>%
  pivot_longer(
    cols = c(Initial, Year1, Year2, Year3),
    names_to = "time_point",
    values_to = "calcium_mg"
  ) %>%
  mutate(
    time_point = factor(time_point,
                       levels = c("Initial", "Year1", "Year2", "Year3"),
                       labels = c("Initial", "Year 1", "Year 2", "Year 3"))
  )

# Step 4: Verify the tidy structure (expect 124 rows × 4 columns).

```

```

cat("Dimensions:", dim(calcium_tidy), "\n")
summary(calcium_tidy)

# Step 5: Compute group means at each time point for the overlay lines.
group_means <- calcium_tidy %>%
  group_by(group, time_point) %>%
  summarize(mean_calcium = mean(calcium_mg, na.rm = TRUE), .groups = "drop")

# Step 6: Plot - faint individual lines underneath bold group mean lines.
# This lets the viewer see both the spread within each group and the
# overall trajectory, without one obscuring the other.
ggplot(calcium_tidy,
  aes(x = time_point, y = calcium_mg,
      group = participant_id, color = group)) +
  geom_line(alpha = 0.22, linewidth = 0.5) +
  geom_line(
    data      = group_means,
    aes(x = time_point, y = mean_calcium,
        group = group, color = group),
    linewidth = 2,
    inherit.aes = FALSE
  ) +
  geom_point(
    data      = group_means,
    aes(x = time_point, y = mean_calcium, color = group),
    size      = 3.5,
    inherit.aes = FALSE
  ) +
  scale_color_manual(
    values = c("Null Treatment" = "#3B8BD4",
              "Diet/Exercise"   = "#1D9E75")
  ) +
  labs(
    title      = "Ulnar Calcium Levels Over Time by Treatment Group",
    subtitle   = "Faint lines = individual participants; bold lines = group means",
    x          = "Time Point",
    y          = "Ulnar Calcium (mg)",
    color      = "Treatment Group"
  ) +
  theme_minimal(base_size = 13) +
  theme(legend.position = "bottom")

library(tidyverse)

```

```

# Step 1: Load the raw file and rename all 8 columns explicitly.
# The first four belong to the null treatment group, the last four to the
# diet/exercise group. Naming them now prevents any ambiguity downstream.
calcium_raw <- read_csv("calcium.csv", col_names = FALSE)

colnames(calcium_raw) <- c(
  "null_initial", "null_year1", "null_year2", "null_year3",
  "dietex_initial", "dietex_year1", "dietex_year2", "dietex_year3"
)

glimpse(calcium_raw) # confirm 31 rows, 8 columns

# Step 2: Separate the two groups into their own data frames, add labels
# and unique participant IDs, then stack them together.
null_group <- calcium_raw %>%
  select(null_initial:null_year3) %>%
  rename(Initial = null_initial, Year1 = null_year1,
         Year2 = null_year2, Year3 = null_year3) %>%
  mutate(participant_id = row_number(),
         group = "Null Treatment")

dietex_group <- calcium_raw %>%
  select(dietex_initial:dietex_year3) %>%
  rename(Initial = dietex_initial, Year1 = dietex_year1,
         Year2 = dietex_year2, Year3 = dietex_year3) %>%
  mutate(participant_id = row_number() + 15, # offset so IDs stay unique
         group = "Diet/Exercise")

calcium_combined <- bind_rows(null_group, dietex_group)

# Step 3: Pivot to long/tidy format. Each row becomes one woman at one
# time point. Factor-order the time column so the plot x-axis is chronological.
calcium_tidy <- calcium_combined %>%
  pivot_longer(
    cols = c(Initial, Year1, Year2, Year3),
    names_to = "time_point",
    values_to = "calcium_mg"
  ) %>%
  mutate(
    time_point = factor(time_point,
                       levels = c("Initial", "Year1", "Year2", "Year3"),
                       labels = c("Initial", "Year 1", "Year 2", "Year 3"))
  )

```



```

# Step 4: Verify the tidy structure (expect 124 rows × 4 columns).

cat("Dimensions:", dim(calcium_tidy), "\n")
summary(calcium_tidy)

# Step 5: Compute group means at each time point for the overlay lines.
group_means <- calcium_tidy %>%
  group_by(group, time_point) %>%
  summarize(mean_calcium = mean(calcium_mg, na.rm = TRUE), .groups = "drop")

# Step 6: Plot - faint individual lines underneath bold group mean lines.
# This lets the viewer see both the spread within each group and the
# overall trajectory, without one obscuring the other.
ggplot(calcium_tidy,
  aes(x = time_point, y = calcium_mg,
      group = participant_id, color = group)) +
  geom_line(alpha = 0.22, linewidth = 0.5) +
  geom_line(
    data = group_means,
    aes(x = time_point, y = mean_calcium,
        group = group, color = group),
    linewidth = 2,
    inherit.aes = FALSE
  ) +
  geom_point(
    data = group_means,
    aes(x = time_point, y = mean_calcium, color = group),
    size = 3.5,
    inherit.aes = FALSE
  ) +
  scale_color_manual(
    values = c("Null Treatment" = "#3B8BD4",
              "Diet/Exercise" = "#1D9E75")
  ) +
  labs(
    title = "Ulnar Calcium Levels Over Time by Treatment Group",
    subtitle = "Faint lines = individual participants; bold lines = group means",
    x = "Time Point",
    y = "Ulnar Calcium (mg)",
    color = "Treatment Group"
  ) +
  theme_minimal(base_size = 13) +
  theme(legend.position = "bottom")

```